



**DuocUC<sup>®</sup>** INFORMÁTICA Y  
TELECOMUNICACIONES



# Deep Learning

---

- Métricas y Generalización

A black and white photograph of a man in a suit holding a tablet, standing in a modern office with glass partitions and a desk in the foreground. The man is smiling and looking at the tablet. The office has a contemporary design with long, horizontal light fixtures and a clean, minimalist aesthetic.

01

# Introducción

# • Introducción

Hasta ahora, solo hemos mencionado la **función de error** de manera general, pero en la práctica obviamente la necesitamos para poder saber que tan bien o mal está el entrenamiento, básicamente la necesitamos para realizar **backpropagation**.

Para los algoritmos de clasificación, lo que nos interesa es saber la diferencia o distancia que existe entre 2 distribuciones de probabilidades.

En nuestro contexto de redes neuronales, vamos a tener 2 distribuciones de probabilidad, **y** e  **$\hat{y}$** .

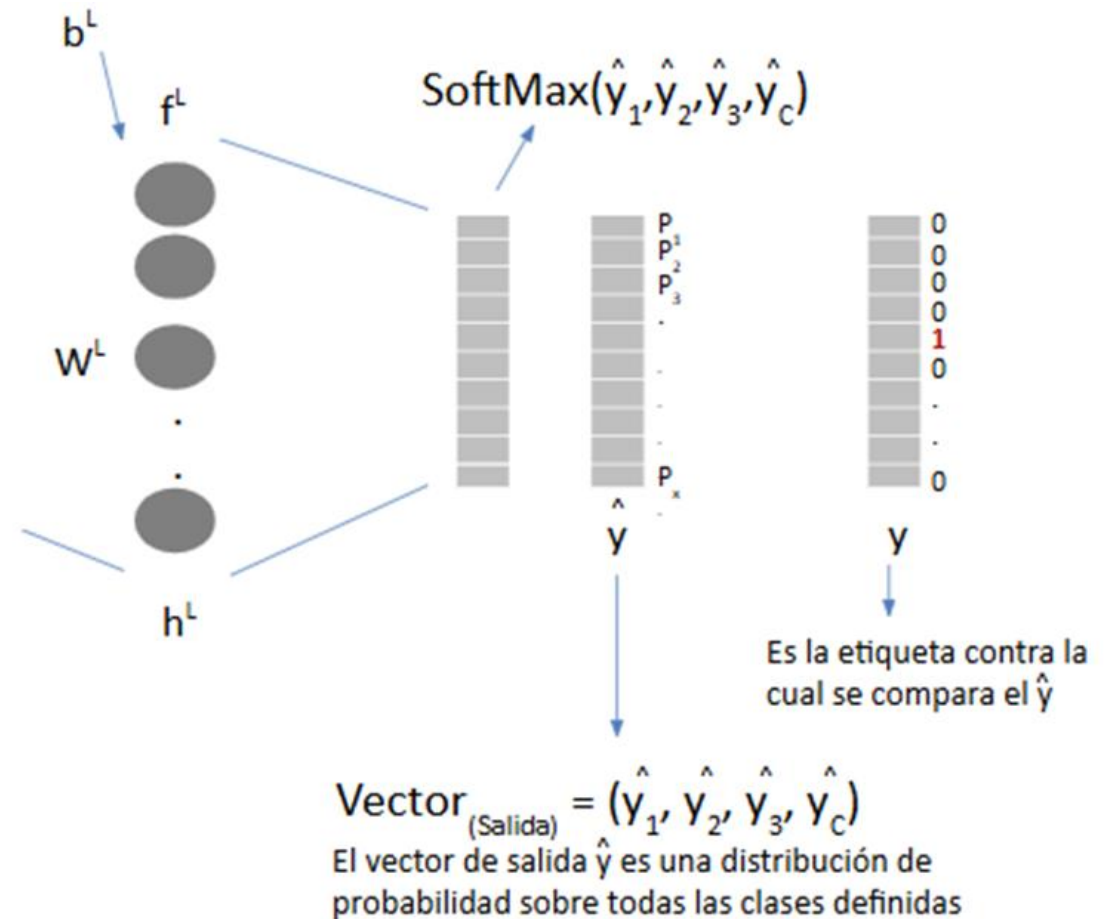
$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \text{error}(\hat{y}^{(i)}; y^{(i)})$$

# • Introducción

Recordemos que, en clasificación, la función de salida recomendada es **Softmax**, la cual genera un vector ( $\hat{y}$ ) tan largo como clases tenga que clasificar y los valores de cada posición representan una probabilidad de que sea esa clase.

Softmax se asegura que cada uno de los valores del vector esté entre 0 y 1 y que la suma de todas sus posiciones no sea  $> 1$  (o 100%).

Por otro lado,  $y$  también es un vector del mismo largo que  $\hat{y}$ , donde todas sus posiciones valen 0, excepto la posición que contiene la clase correcta, la cual se le asigna el valor 1.

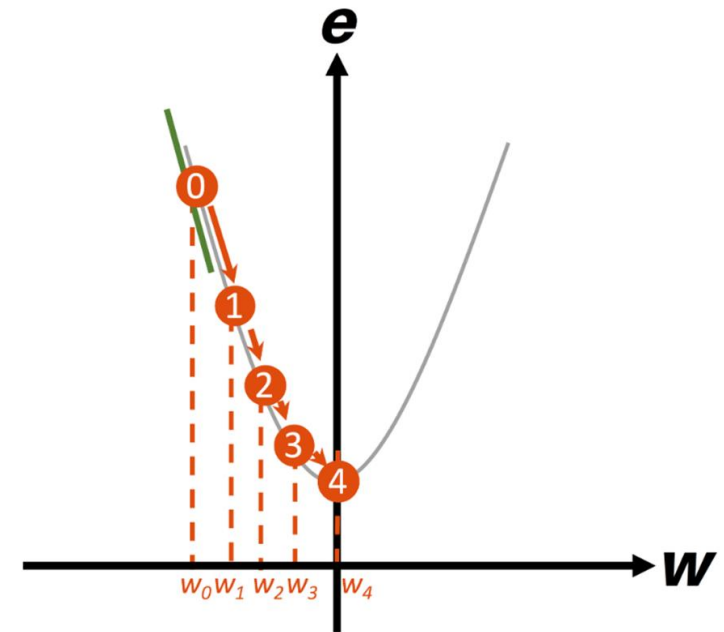


# • Introducción

Recordemos, también, la definición del descenso del gradiente: Pasar todos los ejemplos por la red, calcular el error promedio que la red está cometiendo con de todos los ejemplos y luego usar ese promedio como una función para actualizar los pesos de la red.

El problema con el descenso del gradiente es que la cantidad de ejemplo podrían ser millones o billones y pasar todos estos ejemplos por la red para recién hacer un solo paso del descenso del gradiente es demasiado costoso computacionalmente.

En la práctica, el error se estima usando **SGD (descenso estocástico del gradiente)**: Se escoge un grupo de ejemplos (batch) **al azar**. Este batch de ejemplos se pasa por la red y se calcula el error promedio solo con estos ejemplos. Finalmente se utiliza backpropagation solo con esos valores y descenso del gradiente para actualizar los parámetros, luego se repite cuantas veces se estime necesario.



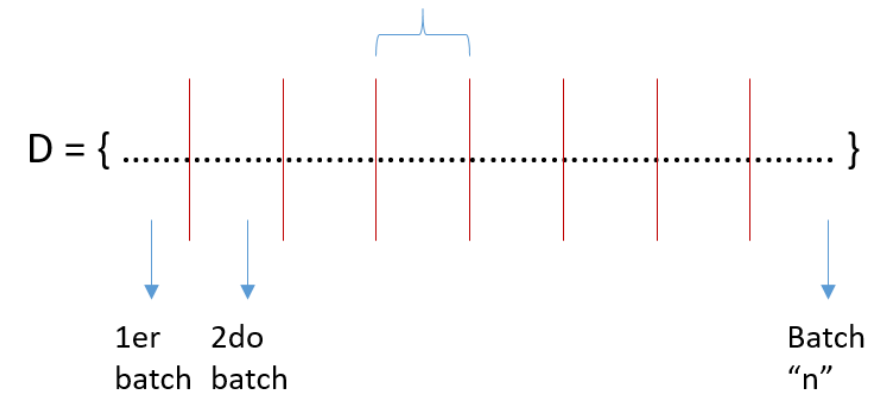
# • Introducción

En rigor, se desordenan todos los ejemplos, se dividen de acuerdo al tamaño del batch elegido, por ejemplo, si hay 100 ejemplos y se escoje un tamaño 5 de batch, se crearan 20 batches de 5 ejemplos.

Una vez que terminados de pasar los 20 batches, se completa 1 iteración o en jerga de Deep learning, **1 época (EPOCH)**.

**Una idea importante y es la razón por la cual es clave saber cómo funcionan las redes neuronales es que en realidad y en general, lo que importa es la cantidad de iteraciones del descenso del gradiente y no la cantidad de épocas necesariamente.**

Tamaño del batch (batch size) es el número de ejemplos en un batch



# • Introducción

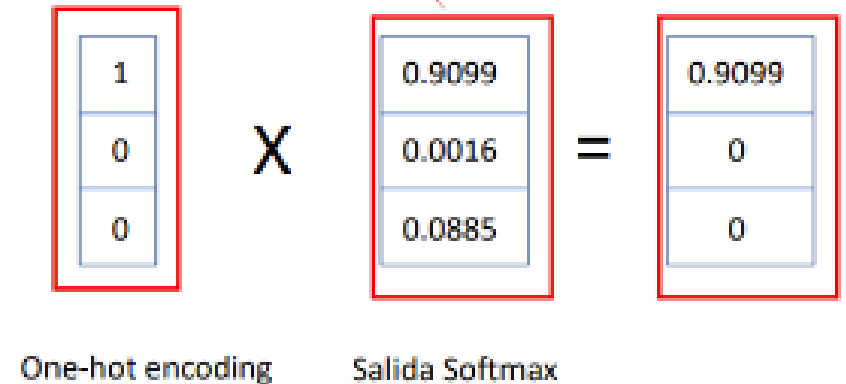
La función de error  $L$  asegura que el entrenamiento sea consistente y bueno.

Para clasificación se usa **CROSSENTROPY**, que compara 2 distribuciones de probabilidades,  $y$  vs  $\hat{y}$  (Para clasificación binaria, suele usarse Sigmoid).

En la práctica,  $y$  es transformado a un vector que representa una distribución de probabilidad (one-hot), donde la posición correcta de la clase en este vector es 1 (100% de probabilidad) y el resto en 0 (0% de probabilidad)

## Cross-entropy Loss

$$L(y, \hat{y}) = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$





02

Métricas

La métrica de desempeño más usada en **Clasificación y Regresión**, es el **Accuracy** (Exactitud)

Se calcula al mismo tiempo que la función de error, es decir, al hacer el forward de una muestra.

Se define como el % total de ejemplos clasificados correctamente.

Funciona muy bien cuando las clases están equilibradas.

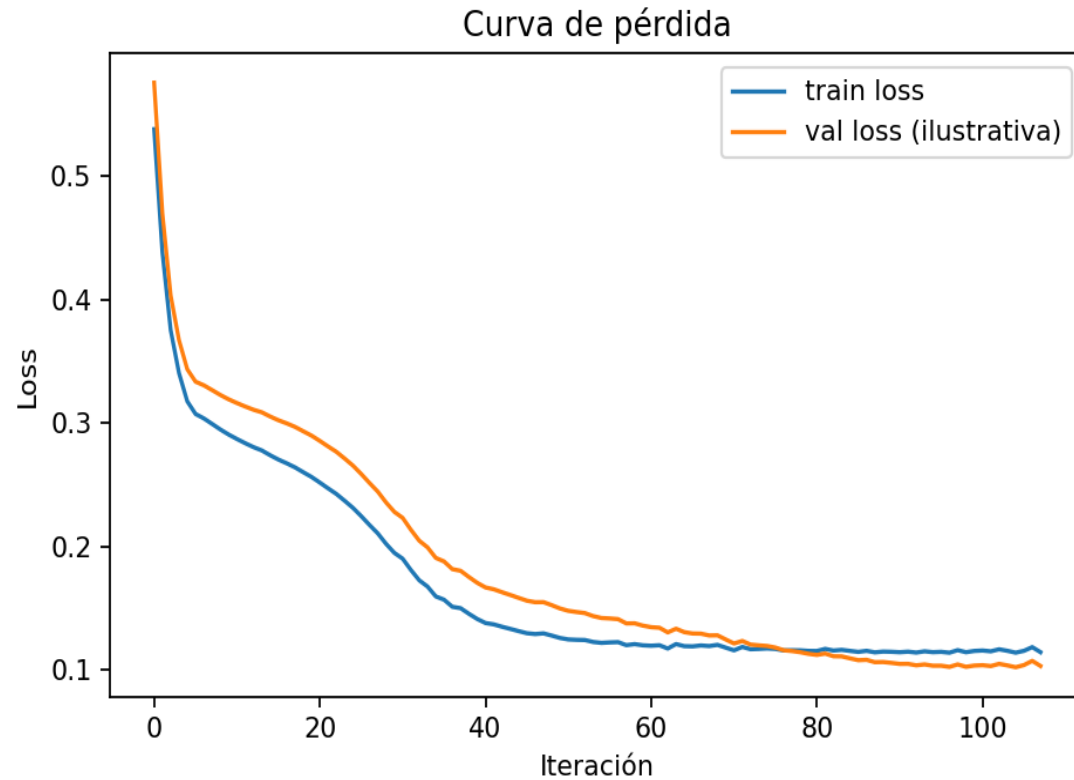
Con esta métrica, se toma la decisión si se continúa o no entrenando la red.

|                    |   | Actual value   |                |
|--------------------|---|----------------|----------------|
|                    |   | P              | N              |
| Prediction outcome | P | True Positive  | False Positive |
|                    | N | False Negative | True Negative  |

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

**Accuracy:** Es el número de verdaderos positivos y verdaderos negativos dividido por el número de verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos.

# Curvas de aprendizaje: primera lectura



## Buen ajuste

Train loss y val loss bajan y se mantienen relativamente cercanas.

## Overfitting

Train loss baja, pero val loss se estanca o sube.

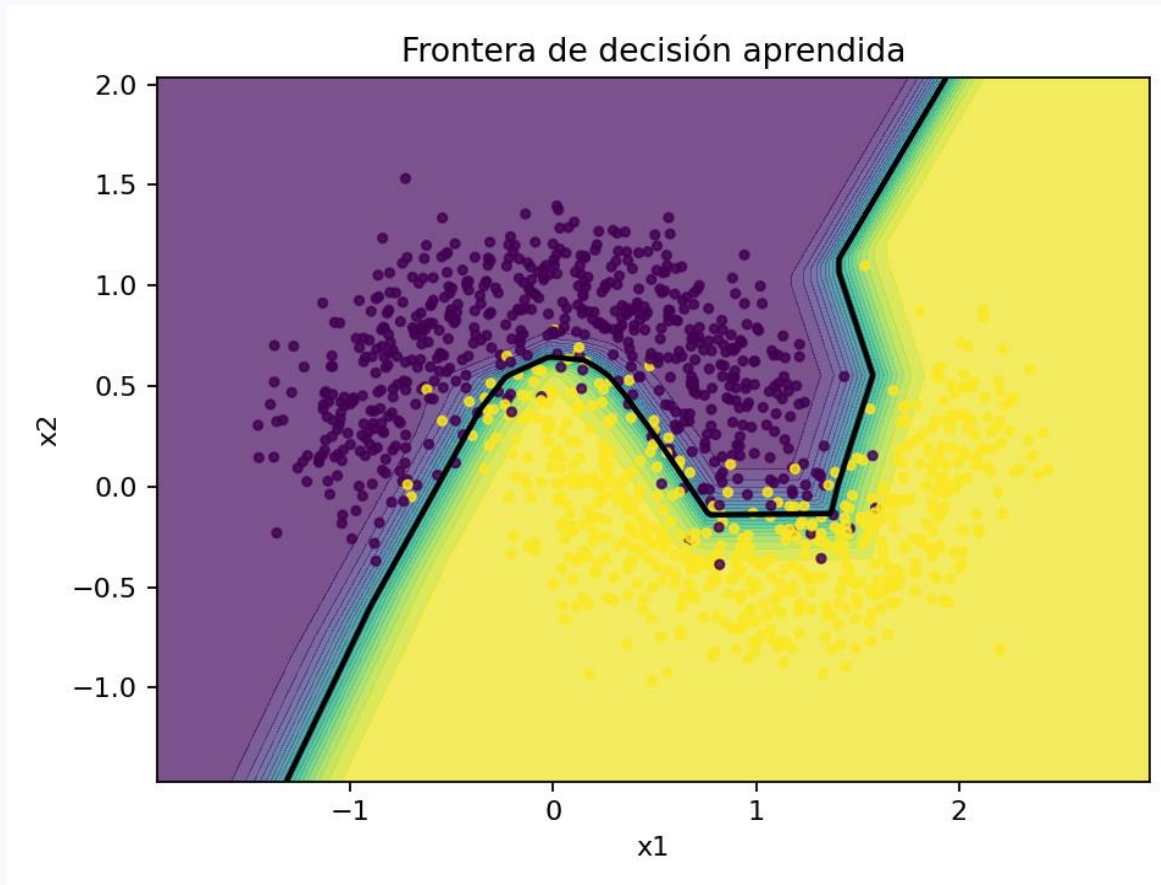
## Underfitting

Ambas pérdidas se mantienen altas o bajan muy poco.

## Uso en clase

Pedir a los estudiantes describir la curva antes de mirar la métrica final.

# Frontera de decisión

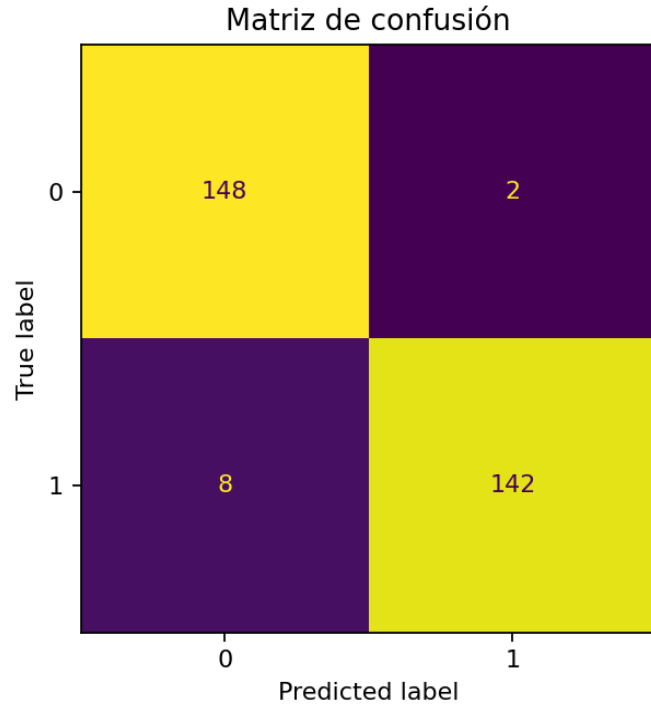


- La zona coloreada muestra la probabilidad de clase 1.
- La línea negra marca el umbral 0.5.
- Una red fully connected puede aprender fronteras no lineales.
- Una frontera demasiado irregular puede indicar sobreajuste al ruido.

## Discusión

Comparar visualmente fronteras de modelos pequeños, medianos y grandes.

# Matriz de confusión



## TN

Real 0 y predicho 0: negativo correctamente clasificado.

## FP

Real 0 y predicho 1: falsa alarma.

## FN

Real 1 y predicho 0: positivo no detectado.

## TP

Real 1 y predicho 1: positivo correctamente detectado.

## Contexto importa

En diagnóstico médico, un falso negativo puede ser crítico. En spam, un falso positivo puede ocultar un correo importante.

# Precision, Recall y F1-score

## Precision

De lo que predije como positivo, ¿cuánto era realmente positivo?

$$TP / (TP + FP)$$

## Recall

De todos los positivos reales, ¿cuántos logré encontrar?

$$TP / (TP + FN)$$

## F1-score

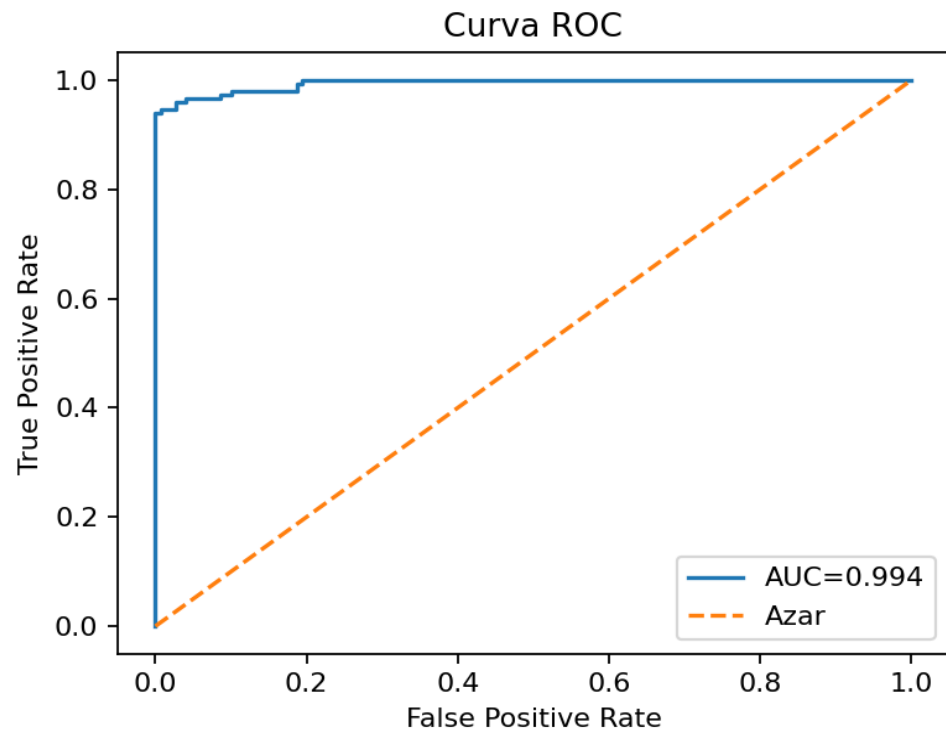
Promedio armónico entre precision y recall. Útil cuando buscamos equilibrio.

$$2PR / (P + R)$$

## Interpretación

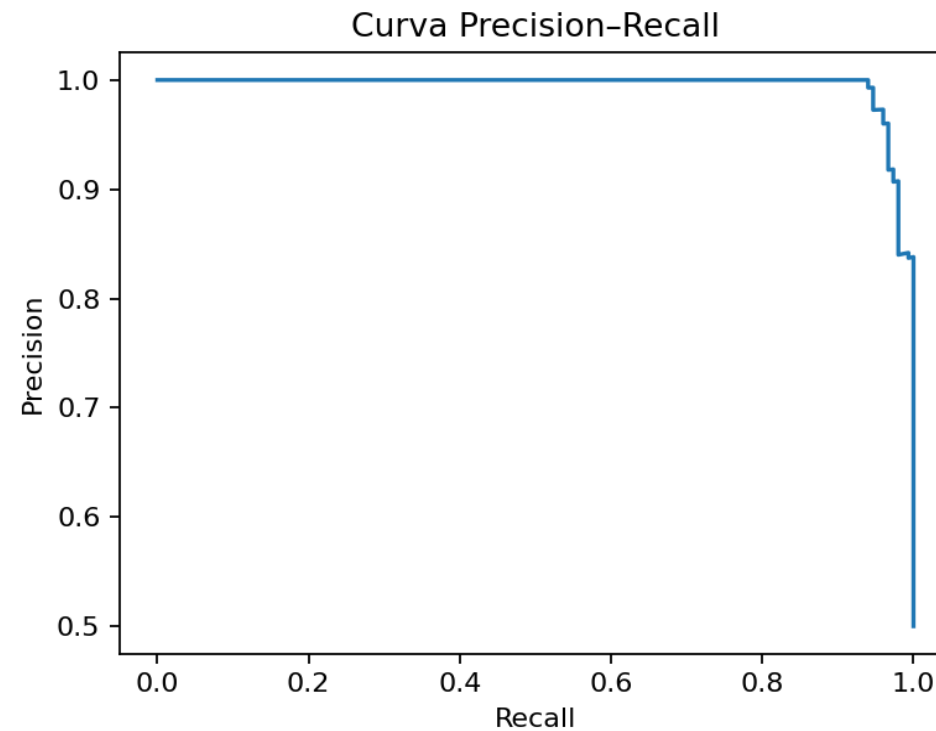
Precision castiga falsas alarmas. Recall castiga positivos que se escapan. F1 resume el balance.

# Curvas ROC y Precision-Recall



## ROC

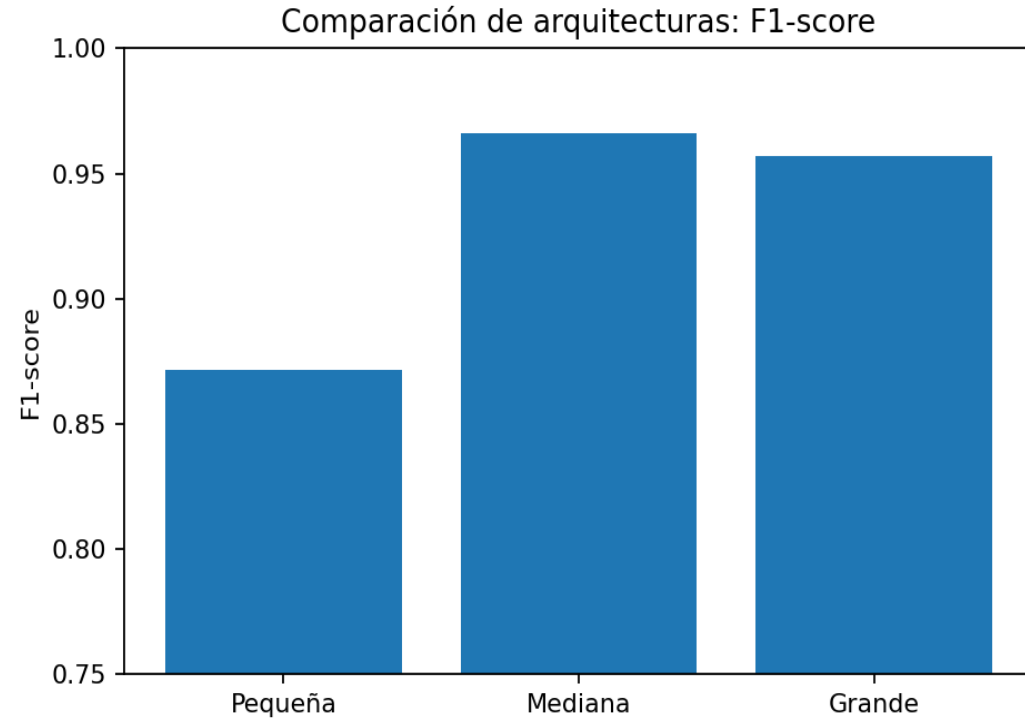
Evalúa sensibilidad vs falsos positivos al mover el umbral. AUC cercano a 1 indica buena separabilidad.



## Precision-Recall

Más informativa cuando hay clases desbalanceadas o interesa la clase positiva.

# Comparación de arquitecturas



## Pequeña: (2)

Poca capacidad. Riesgo de underfitting: frontera demasiado simple.

## Mediana: (8,8)

Buen punto inicial: capacidad suficiente y fácil de entrenar.

## Grande: (32,32,16)

Mayor capacidad. Puede mejorar, pero también memorizar ruido.

## Pregunta

¿Qué métrica conviene mirar si la clase positiva es la más importante?

# Métricas de regresión

## MAE

Promedio del error absoluto. Fácil de interpretar porque está en la unidad original.

## MSE

Promedio del error al cuadrado. Penaliza fuertemente errores grandes.

## RMSE

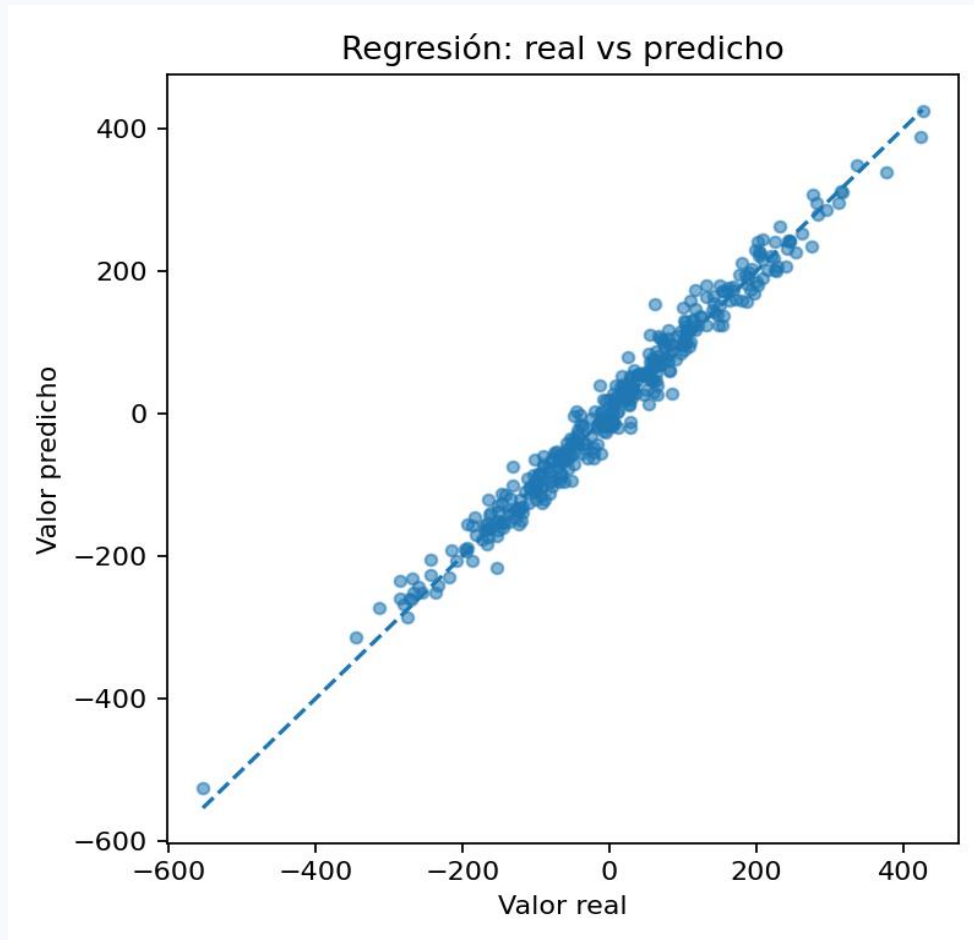
Raíz del MSE. Vuelve a la unidad original y mantiene penalización a errores grandes.

## $R^2$

Proporción de variabilidad explicada por el modelo. 1 es perfecto; 0 equivale a predecir el promedio; menor que 0 es peor.

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

# Real vs predicho

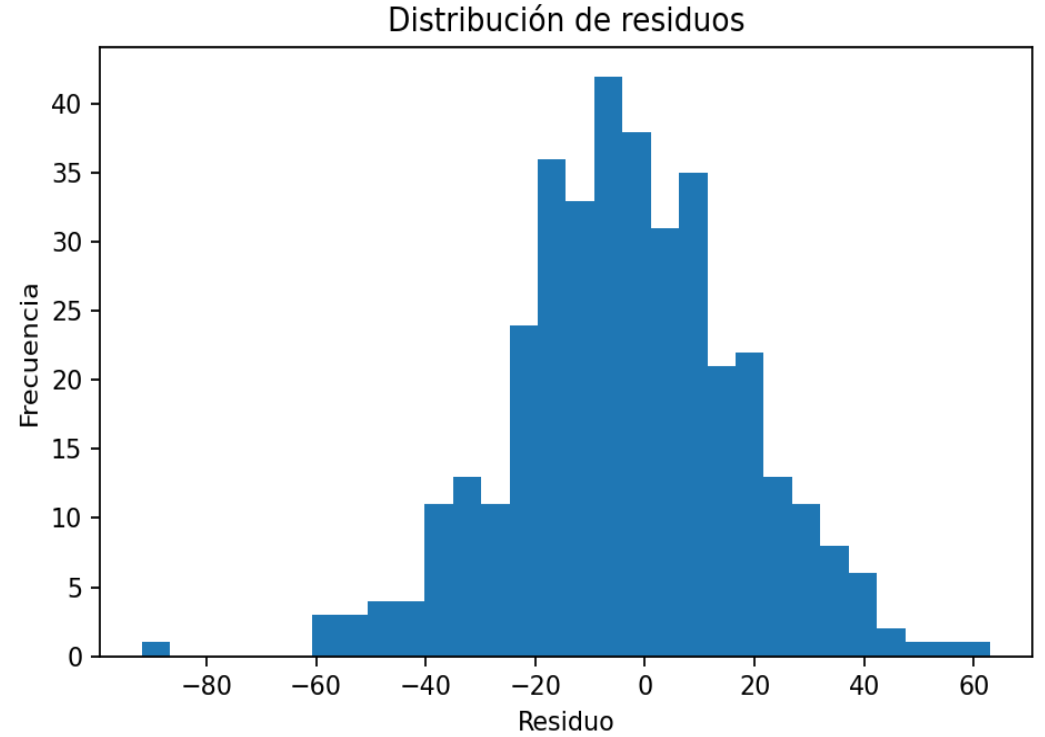
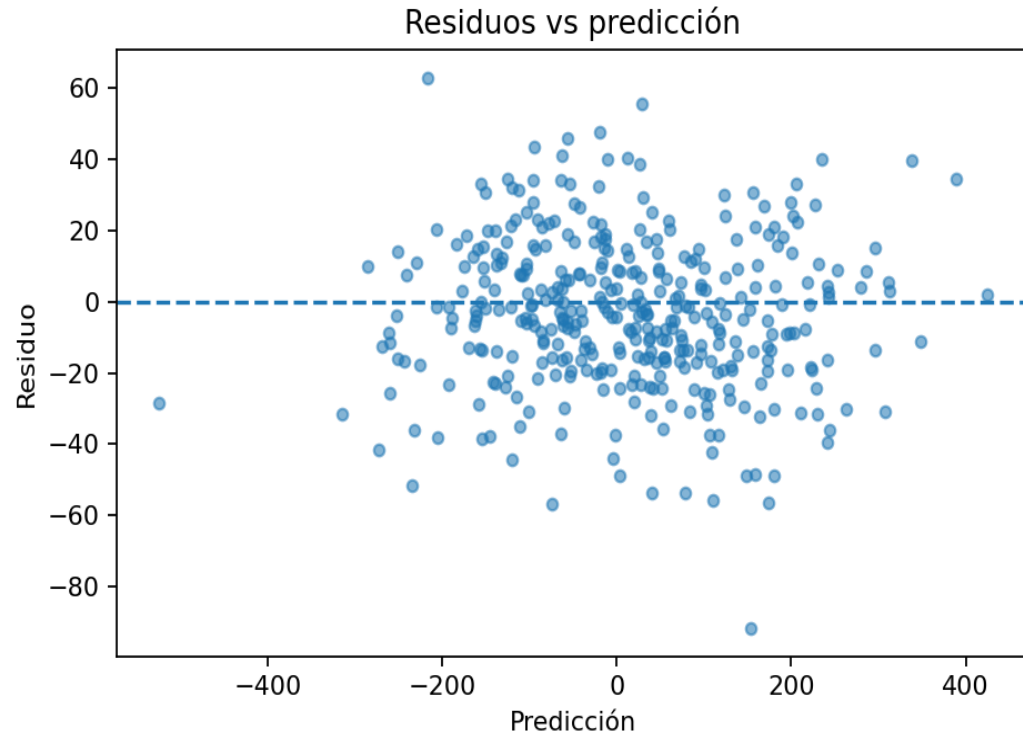


- La diagonal representa predicción perfecta.
- Puntos cerca de la diagonal indican bajo error.
- Dispersión amplia indica errores altos.
- Patrones curvos sugieren que el modelo no captura bien la relación.
- Outliers pueden dominar MSE y RMSE.

## Actividad

Pedir a los estudiantes estimar visualmente si el  $R^2$  será alto, medio o bajo.

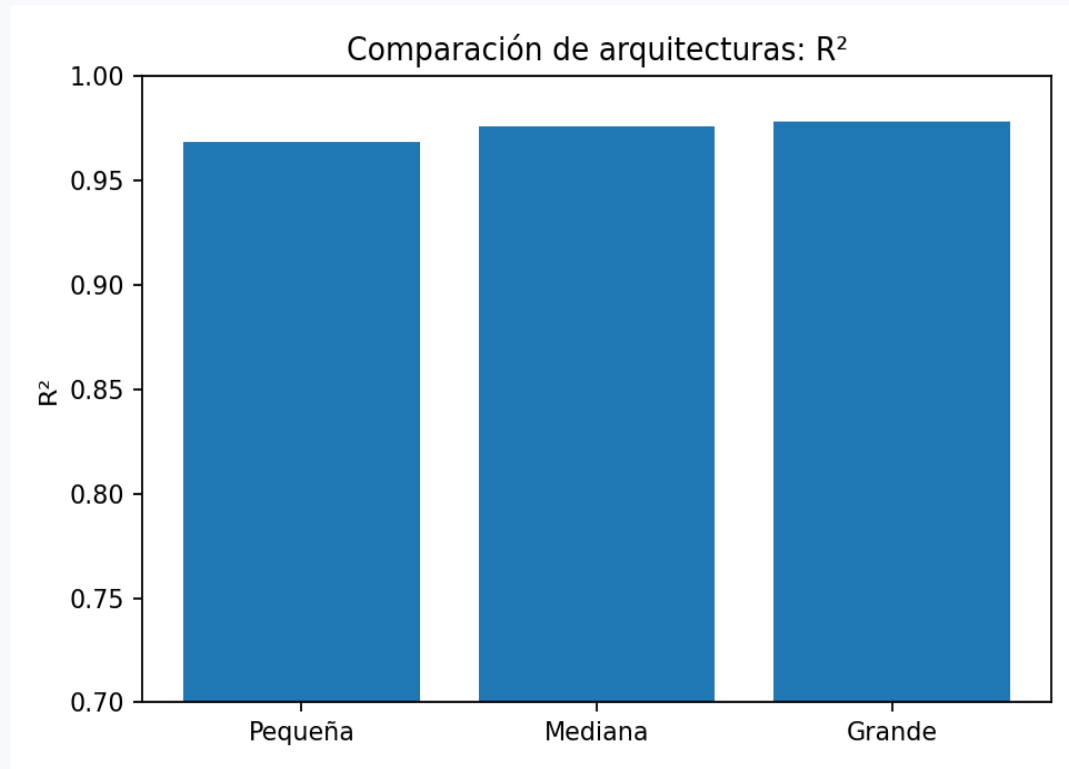
# Análisis de residuos



**Residuo =  $y$  real –  $y$  predicho**

Idealmente, los residuos se distribuyen alrededor de cero y sin patrón sistemático.

# Comparación de arquitecturas en regresión



## R<sup>2</sup> alto

El modelo explica gran parte de la variabilidad de la variable objetivo.

## R<sup>2</sup> cercano a 0

El modelo no supera claramente predecir el promedio.

## R<sup>2</sup> negativo

El modelo es peor que una predicción promedio constante.

## Cuidado

Un R<sup>2</sup> alto no reemplaza revisar residuos: puede ocultar errores sistemáticos en ciertos rangos.



03

**Generalización**

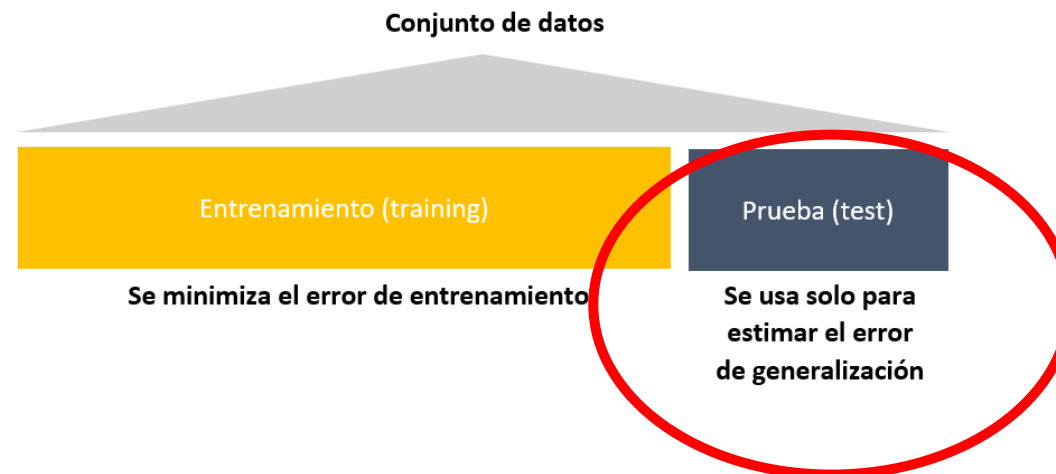
# Generalización

Al ajuste de los parámetros ( $W$ ,  $b$ ) se le conoce como **Optimización**, usando TRAINING SET (o CONJUNTO DE DATOS DE ENTRENAMIENTO).

La **Generalización** es el error promedio que la red comete en datos que **NO** son de entrenamiento, es decir, en datos que nunca haya visto, usando TESTING SET

En Deep Learning, los datos de test NO se pueden usar para el entrenamiento y se debe cuidar que estos sean tan parecidos como se pueda a los que se usaran en la realidad.

En Deep Learning, lo más importante no es el error de entrenamiento obtenido, sino que es el error de generalización. **El error promedio que comete la red con los datos de test.**



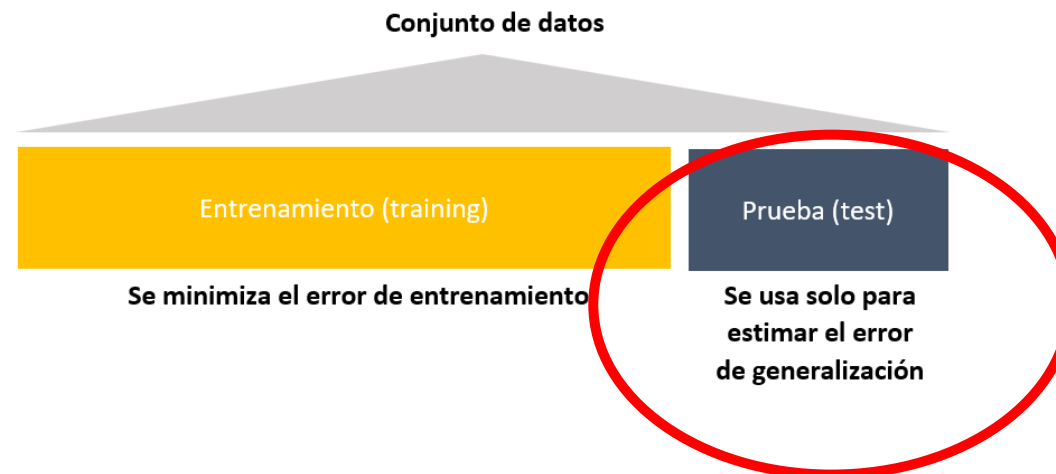
# Generalización

Para crear una buena generalización, en general hay que **estresar** la red.

Ejemplo, si existe un conjunto de datos de mala calidad y otro de buena de calidad, se entrena con el de mala calidad y se prueba con el bueno.

La motivación es que los datos de test sean lo más parecido a la realidad, lo que se va a encontrar cuando el modelo esté en producción.

La **capacidad** tiene que ver con todas las posibles funciones que la red será capaz de aprender, es decir, al crear una red, se define y fija la cantidad de neuronas y la cantidad de capas. Cada vez que los parámetros son ajustados, se crea una nueva red – la misma arquitectura – pero una nueva red. Hay una cantidad fija de funciones que se podrán aprender.



# Generalización

La capacidad de una red se controla (aumenta o disminuye) con los hiperparámetros:

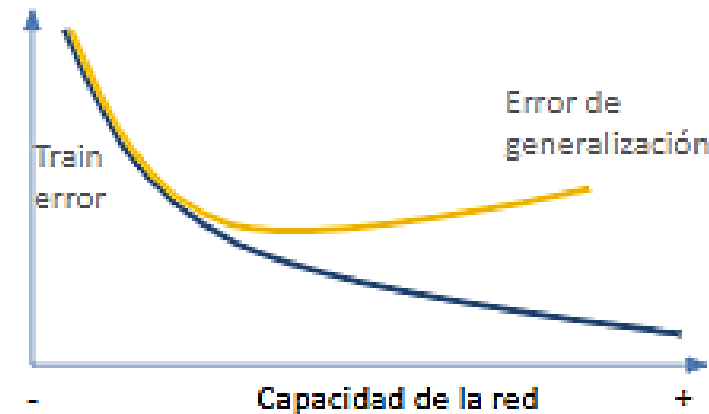
- Más cantidad de capas
- Más cantidad de neuronas por capas
- Tasa de aprendizaje
- Tiempo entrenamiento (cant. épocas)
- Funciones de activación
- Tamaño del batch
- Regularización

Hay que tener un cuidado especial, en general, cuando se le da mayor capacidad a la red.

Baja el error de entrenamiento, pero aumenta el error de generalización.

Finalmente, es el error de generalización el que interesa minimizar.

Se puede caer en **overfitting** y **underfitting**



A mayor capacidad de la red, menor es el error de training

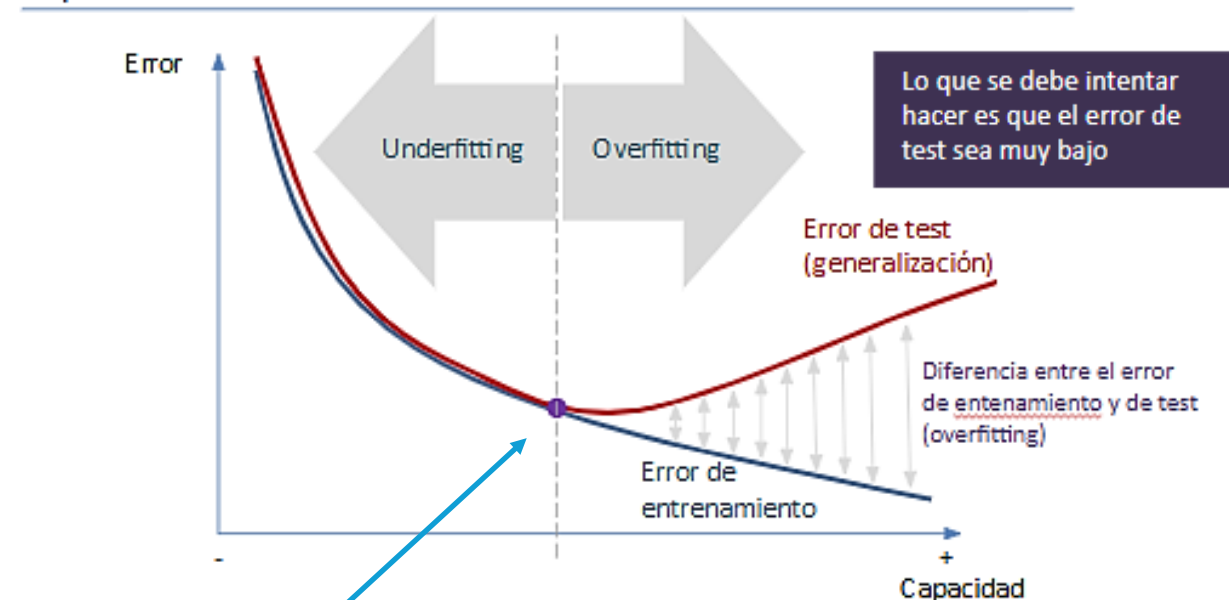
# Generalización

**Overfitting es cuando la diferencia entre el error de entrenamiento y el error de test es muy grande** (Error de entrenamiento baja y Error de test sube), En otras palabras, cuando se observa el error con un dato que no se está entrenando, el error es más alto

**Underfitting es cuando el error de entrenamiento es muy grande.** Con redes neuronales, “nunca” se debería estar en underfitting, a menos que los datos sean muy, pero muy malos (ejemplo, pésimamente etiquetados).

En general, **siempre el error de entrenamiento es más bajo que el error de test.**

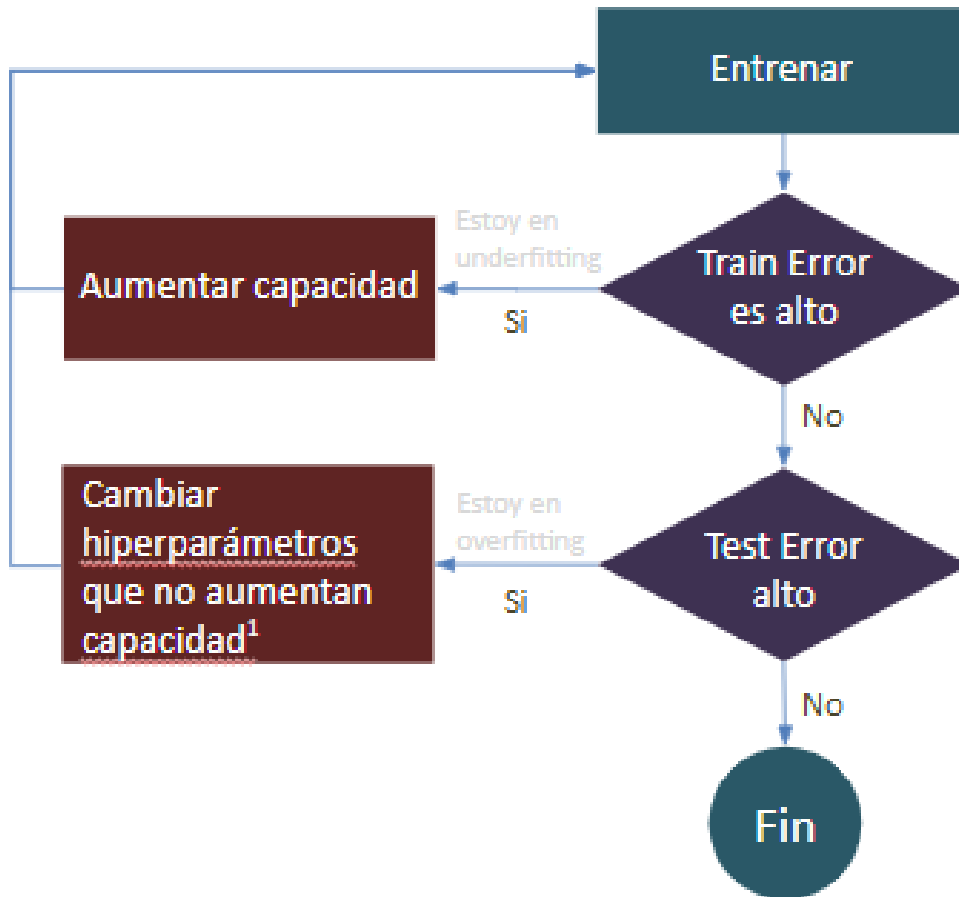
Capacidad versus error



Capacidad óptima de la red es cuando el error de train es muy parecido al error de test  
(esto es lo que se debe tratar de encontrar)

# • En resumen

Pasos para mejorar la Generalización de una red.



• Este “loop” podría ser caro (cómputo, dinero (por los datos adicionales a conseguir), tiempo para entrenar)

• En general, se debería tender a una red con gran capacidad (tener un modelo tan grande como pueda).

• Se debe reducir la diferencia entre el error de entrenamiento y el error con los datos o ejemplos de test.

# DuocUC<sup>®</sup>



CERCANÍA. LIDERAZGO. FUTURO.

**7**  
AÑOS  
ACREDITADO



**NIVEL DE EXCELENCIA**  
HASTA OCTUBRE 2031

Docencia de pregrado / Gestión institucional / Aseguramiento interno de la calidad / Vinculación con el Medio / Investigación, creación y/o innovación